WebGPU for Intensity Diffraction Tomography: A New Method for 3D Imaging

Rayan Syed[1], Andrew Xin[1], and Lei Tian[1]

[1]Department of Electrical and Computer Engineering, Computational Imaging Systems Lab, Boston University
rsyed@bu.edu, andrewx@bu.edu, leitian@bu.edu

Code: https://github.com/bu-cisl/wgpu-idt Demo: https://bu-cisl.github.io/wgpu-idt/

Abstract

This report presents a WebGPU-based framework for forward and inverse modeling in intensity diffraction tomography, with an emphasis on accessibility and deployability of computational imaging systems. The proposed approach enables physics-based wave propagation models, including the split-step non-paraxial method, beam propagation method, and Born approximation, to be executed in a portable, browser-compatible environment without reliance on specialized GPU frameworks.

The system implements both forward simulation and an iterative reconstruction pipeline for recovering three-dimensional refractive index volumes from intensity-only measurements. Experimental results demonstrate that the WebGPU implementation achieves high numerical accuracy and stable reconstruction behavior, while exhibiting expected scaling characteristics relative to a CUDA-based baseline.

Although performance remains limited compared to highly optimized native implementations, the framework significantly lowers the barrier to entry for computational imaging workflows and enables new use cases in education, reproducibility, and interactive exploration. These results demonstrate that complex physics-based imaging pipelines can be effectively deployed in hardware-agnostic, web-based environments.

1 Introduction

Intensity Diffraction Tomography (IDT) is a computational imaging technique that enables the reconstruction of a three-dimensional (3D) refractive index (RI) distribution from a series of two-dimensional (2D) intensity measurements acquired under varying illumination conditions [1]. By leveraging physics-based models of light propagation and scattering, IDT provides a label-free and cost-effective alternative to traditional volumetric microscopy methods, with applications in biological imaging and materials science.

In a typical IDT setup, a sample is illuminated by plane waves at different angles, and the resulting intensity measurements are captured by a camera [1]. Each measurement encodes partial information about the underlying 3D structure. The goal of IDT is to invert this measurement process and recover the underlying RI distribution from the observed data.

This inverse problem is inherently challenging due to its nonlinear and ill-posed nature. It is commonly addressed by formulating reconstruction as an optimization problem, where a forward model is repeatedly evaluated and compared to measured data. This formulation closely mirrors machine learning training pipelines, where a differentiable model is optimized with respect to a loss function using iterative gradient-based methods.

Despite its advantages, existing IDT implementations, especially those based on high-fidelity multiple-scattering models such as the Split-Step Non-Paraxial (SSNP) method [2], typically rely on specialized GPU frameworks such as CUDA. These dependencies limit accessibility, reproducibility, and ease of deployment across diverse computing environments.

Furthermore, deploying existing GPU-based implementations in widely accessible environments such as the browser is challenging. Common machine learning and GPU frameworks are either not designed for direct compilation to web-compatible targets or require server-side execution, introducing additional infrastructure complexity and cost.

In this work, we present a WebGPU-based framework for scalable forward and inverse modeling in IDT. By leveraging modern browser-compatible GPU compute capabilities [3], the proposed system enables hardware-agnostic execution of computational imaging pipelines. The framework implements multiple forward models, including SSNP, Beam Propagation Method (BPM) [4], and the Born approximation [5], alongside an iterative reconstruction procedure for recovering 3D RI volumes.

The contributions of this work are threefold. First, we develop a modular WebGPU-based implementation of key wave propagation models for IDT. Second, we implement an inverse reconstruction pipeline using gradient-based optimization. Third, we empirically evaluate the forward models and assess the reconstruction pipeline using controlled synthetic experiments, analyzing reconstruction quality and convergence behavior.

2 Background and Problem Formulation

We now formalize the forward imaging model and inverse reconstruction problem underlying the system implemented in this work. Figure 1 provides an overview of the imaging setup, forward propagation model, and iterative reconstruction pipeline.

2.1 Forward Model

Let $\mathbf{n}(\mathbf{r})$ denote the RI distribution of the sample, where $\mathbf{r} = (x, y, z)$ is the spatial coordinate. Under the k-th illumination condition, the sample is excited by an incident field $U_{\text{in}}^{(k)}$, which is

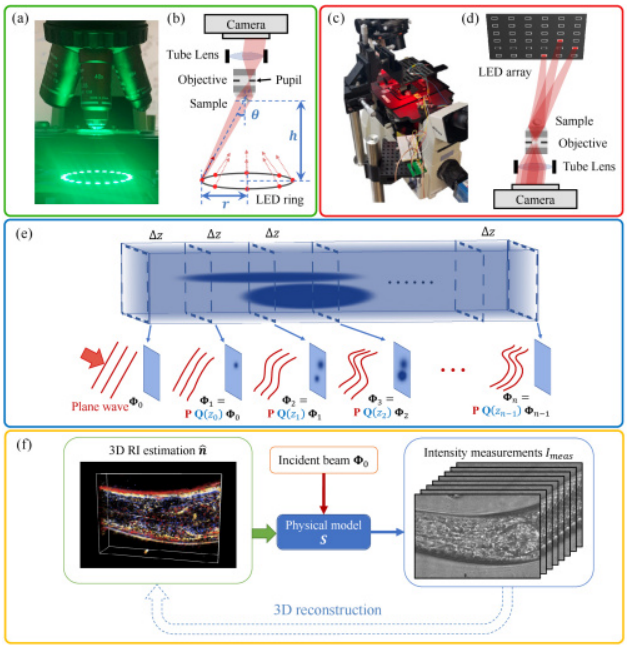

Figure 1: Overview of the IDT system and reconstruction pipeline, adapted from [2]. (a-b) Annular illumination using an LED ring produces angle-dependent measurements. (c-d) LED array configurations enable flexible illumination patterns. (e) The forward model (like SSNP) alternates between propagation and scattering operations across axial slices. (f) Reconstruction is performed by iteratively solving an optimization problem to recover the 3D RI distribution.

typically modeled as a plane wave:

$$U_{\text{in}}^{(k)}(\mathbf{r}) = \exp\left(i\mathbf{k}_k \cdot \mathbf{r}\right), \tag{1}$$

where $\mathbf{k}_k$ is the wavevector corresponding to illumination angle ϕ_k. Here, ϕ_k parameterizes the direction of incidence, typically defined by transverse components (k_x, k_y) determined by the illumination geometry.

As the incident field propagates through the sample, it is transformed into a scattered complex field $U_{\text{out}}^{(k)}$ at the sensor plane. The measured intensity is given by

$$I_{\text{meas}}^{(k)} = \left|U_{\text{out}}^{(k)}\right|^2, \tag{2}$$

where $U_{\text{out}}^{(k)} \in \mathbb{C}^{H\times W}$ denotes the complex optical field and H, W are the sensor dimensions.

The forward model can be written compactly as

$$U_{\text{out}}^{(k)} = \mathcal{S}(\mathbf{n}; \phi_k), \tag{3}$$

where $\mathcal{S}$ is a nonlinear operator that models both scattering within the sample and propagation to the detector. The intensity measurement is obtained only after taking the squared magnitude of this complex field, which removes phase information and makes the inverse problem more challenging.

In this work, we consider three approximations of the operator $\mathcal{S}$. The Born approximation assumes weak scattering and linearizes the interaction between the incident field and the object [5]. The BPM models paraxial wave evolution through the sample by alternating between phase

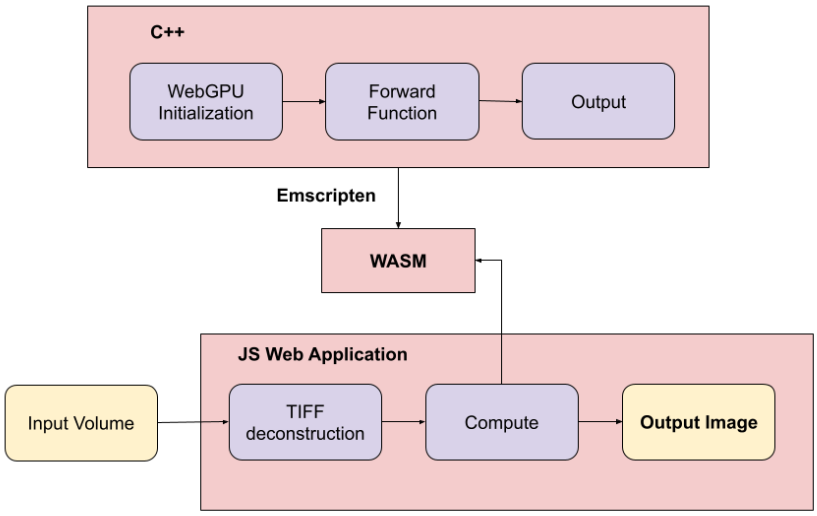

Figure 2: System architecture of the WebGPU-based framework. A C++ backend built on WebGPU is compiled via Emscripten to WebAssembly, enabling both native execution and browser-based deployment.

modulation and free-space propagation [4]. The SSNP method provides a higher-fidelity multiple-scattering model by propagating the field through discrete axial slices while accounting for non-paraxial effects [2]. As illustrated in Figure 1(e), this process involves successive propagation and scattering updates that capture complex wave interactions within the sample.

2.2 Inverse Problem

Given a set of intensity measurements $\{I_{\text{meas}}^{(k)}\}$, reconstruction is formulated as an optimization problem over the RI distribution:

$$\min_{\mathbf{n}} \sum_{k} \left\| \sqrt{I_{\text{pred}}^{(k)}(\mathbf{n})} - \sqrt{I_{\text{meas}}^{(k)}} \right\|_2^2, \tag{4}$$

where $I_{\text{pred}}^{(k)}$ is obtained by evaluating the forward model under illumination ϕ_k.

This formulation requires differentiating through the forward operator, which in practice involves propagating gradients backward through the sequence of scattering and propagation steps shown in Figure 1(e). The resulting reconstruction algorithm iteratively updates the estimate of $\mathbf{n}$ using gradient-based optimization, repeatedly invoking the forward model and minimizing the discrepancy between predicted and measured amplitudes.

As illustrated in Figure 1(f), this process produces a 3D RI reconstruction that is consistent with the observed measurements.

3 Implementation

This section describes the system architecture and implementation of the proposed framework.

The core of the system is a C++ backend built on top of WebGPU using a WebGPU C++ wrapper [3]. This provides a lightweight interface to the WebGPU API while preserving direct

control over memory and execution. The backend handles all GPU operations from handling device selection to orchestrating compute shader files and managing buffer communication. As shown in Figure 2, the backend code is compiled using Emscripten to WebAssembly, allowing the same codebase to run both as a native executable and within a browser environment.

On the frontend side, a JS web application interface allows for uploading of .tiff files with volume data and carefully decodes volume data into a .bin file for the C++ executable to process. The final result is then rendered to the screen for users. All of this enables hardware-agnostic GPU execution and improves accessibility compared to CUDA-based implementations.

Forward simulation is implemented as a sequence of GPU compute operations. The RI volume is represented as a 3D grid, and wave propagation is computed slice-by-slice along the axial direction. Each step consists of phase modulation in the spatial domain followed by propagation in the Fourier domain. The implementation supports multiple forward models corresponding to different approximations of the scattering operator, including the Born approximation, the BPM, and the SSNP method. These models share the same underlying GPU primitives, enabling a modular design in which different propagation strategies can be interchanged without modifying the surrounding pipeline.

The reconstruction module extends the original forward-only implementation by introducing an iterative optimization procedure over the RI volume. Starting from an initial estimate, the algorithm repeatedly evaluates the forward model, computes the discrepancy with observed measurements, and updates the volume using gradient-based optimization. Gradients are computed by explicitly propagating adjoint fields backward through the same sequence of operations used in the forward pass, reversing propagation steps and accumulating updates at each axial slice. This ensures that both forward and inverse computations are executed entirely on the GPU.

For evaluation, Python scripts are used to generate synthetic volumes, construct illumination angle sets, and orchestrate reconstruction runs. These scripts also provide a reference implementation using PyTorch, enabling direct comparison between the WebGPU-based implementation and baseline methods.

4 Evaluation

We evaluate the proposed framework along two dimensions: correctness of the forward models and the performance of the reconstruction pipeline. We begin by validating the accuracy of the forward simulation.

4.1 Forward Model Evaluation

To verify correctness, we compare the outputs of the WebGPU-based implementation against a PyTorch reference implementation for each forward model. All comparisons are performed using identical input volumes and illumination conditions, and errors are measured in terms of relative differences between predicted intensity fields.

4.1.1 Accuracy

We find that the WebGPU implementation produces results consistent with the reference implementation across all three forward models. For sufficiently large input volumes, such as $2048 \times 2048 \times 256$, the relative error remains within a tolerance of 10^{-4} for the Born approximation, BPM, and SSNP models.

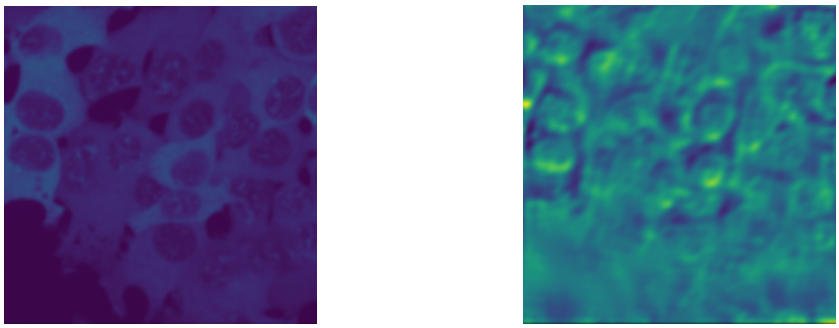

Figure 3: Qualitative example of the forward imaging process. Left: example sample structure (illustrative). Right: simulated intensity measurement produced by our SSNP forward model under a given illumination condition.

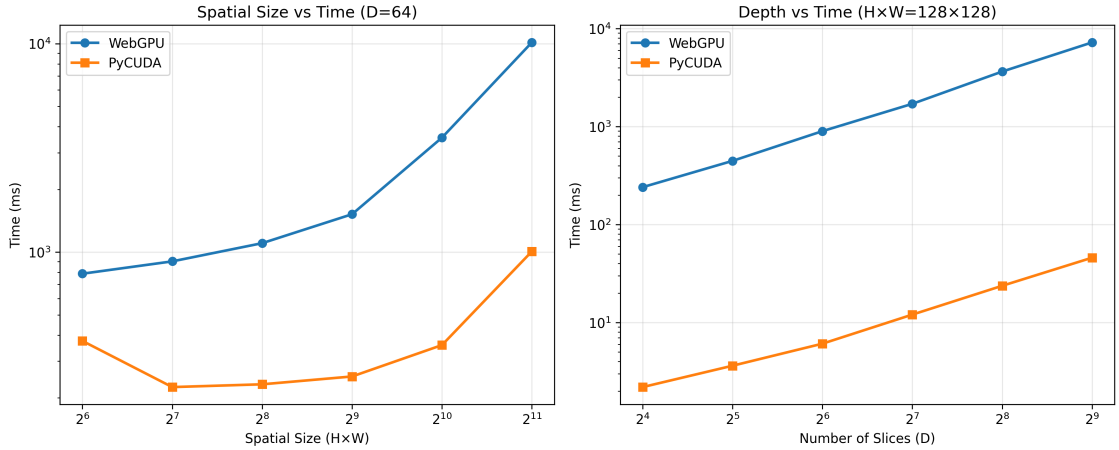

Figure 4: Runtime comparison between WebGPU and PyCUDA implementations of the SSNP forward model. Left: scaling with spatial resolution ($H \times W$) at fixed depth $D = 64$. Right: scaling with depth (D) at fixed spatial resolution 128×128. Both axes are plotted on a logarithmic scale.

This level of agreement along with a manual inspection as seen in Figure 3 indicate that the WebGPU-based implementation accurately reproduces the underlying wave propagation behavior while maintaining numerical stability at scale. Minor discrepancies are attributed to floating-point differences and implementation-specific details in GPU execution, but do not significantly affect the resulting intensity fields.

4.1.2 Performance

We evaluate the runtime performance of the WebGPU-based forward model against a PyCUDA implementation using the SSNP model. We focus exclusively on SSNP, as it represents the most computationally demanding forward model among those implemented, incorporating multiple-scattering effects and per-slice propagation. As such, it provides the most representative benchmark for assessing system-level performance. The Born approximation and BPM involve simpler propagation assumptions and exhibit strictly lower computational cost, and therefore are omitted from detailed performance analysis.

When varying spatial resolution while keeping depth fixed ($D = 64$), both implementations

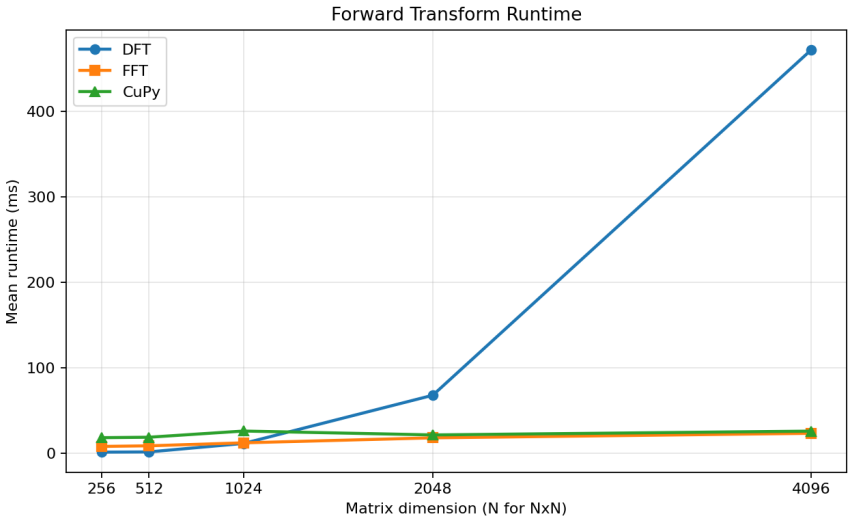

Figure 5: Runtime comparison of DFT and FFT implementations. The DFT exhibits quadratic scaling, while the FFT achieves significantly improved performance comparable to optimized GPU libraries such as CuPy.

exhibit similar scaling trends as the image size increases, as shown in Figure 4. However, the WebGPU implementation remains consistently slower than the PyCUDA baseline across all tested resolutions. This gap grows with problem size, indicating that while the underlying computational structure is consistent, differences in execution efficiency accumulate at larger scales.

When varying depth while keeping spatial resolution fixed ($H \times W = 128 \times 128$), the runtime increases approximately linearly for both implementations, reflecting the per-slice propagation structure of the forward model. In this setting, the performance gap becomes more pronounced, with the WebGPU implementation incurring significantly higher runtime as the number of slices increases.

Initially, we hypothesized that this slowdown was primarily due to the use of a naive Discrete Fourier Transform (DFT) in the WebGPU implementation. To investigate this, we benchmarked the DFT and a Fast Fourier Transform (FFT) implementation in isolation, as shown in Figure 5. As expected, the DFT exhibits significantly worse scaling behavior, while the FFT achieves performance comparable to optimized GPU libraries such as CuPy.

However, integrating the FFT into the full forward pipeline did not substantially improve end-to-end runtime. This indicates that, despite the theoretical importance of efficient transform operations, the forward model is not transform-bound in practice. Instead, performance is dominated by other factors, including memory movement, per-slice computation, and kernel dispatch overhead.

These results suggest that further optimization efforts should focus on improving memory access patterns and reducing execution overhead, rather than solely optimizing individual computational primitives such as the Fourier transform.

4.2 Reconstruction Evaluation

We evaluate the reconstruction pipeline using synthetic measurements generated from a known RI volume. The target volume is defined on a $50 \times 50 \times 50$ grid with spatial resolution $(0.2, 0.2, 0.2)$ μm, corresponding to a $10 \times 10 \times 10$ μm region of interest. Within this volume, the sample is

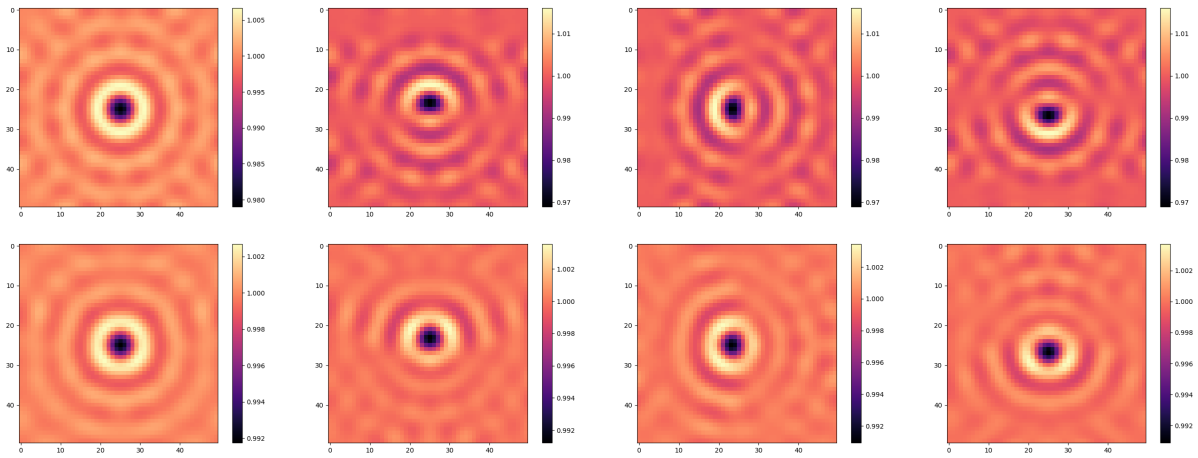

Figure 6: Qualitative reconstruction results evaluated at four illumination conditions. Top row: forward simulations from the ground truth volume. Bottom row: forward simulations from the reconstructed volume. From left to right, the illumination angles are $(0, 0)$, $(0.12, 0.00)$, $(0.00, 0.12)$, and $(-0.12, 0.08)$.

modeled as a single spherical bead centered in the domain with radius 2.5 voxels and RI contrast value 0.02. Reconstruction is initialized from an all-zero volume of the same size.

For each experiment, an intensity stack is constructed using 15 illumination angles arranged on a ring together with a central $(0, 0)$ illumination condition, yielding a total of 16 measurements. Reconstruction is then performed by iteratively optimizing the RI volume to match this measured intensity stack.

To assess reconstruction quality, we evaluate the recovered volume by comparing simulated forward measurements at a small set of representative illumination conditions. Specifically, four angles are arbitrarily selected for qualitative inspection: $(0, 0)$, $(0.12, 0.00)$, $(0.00, 0.12)$, and $(-0.12, 0.08)$. These angles are chosen as a consistent subset of the illumination space for side-by-side comparison between the original and reconstructed volumes. The reconstruction is performed using gradient-based optimization with a fixed learning rate of 5×10^{-1}.

As shown in Figure 6, the reconstructed volume produces forward measurements that are visually consistent with those generated from the ground truth volume across the selected illumination conditions. Structural features are preserved, and the reconstruction converges to a solution that captures the overall geometry of the bead.

The reconstruction process results in a consistent reduction in measurement error, with the mean squared error decreasing from 1.38×10^{-5} to 9.60×10^{-6} over 298 iterations, indicating stable convergence of the optimization procedure.

However, a consistent discrepancy is observed in the amplitude of the reconstructed measurements. While spatial structure is recovered reasonably well, the reconstructed intensity values are systematically attenuated relative to the ground truth. This indicates that the optimization successfully matches the measurement patterns but does not fully recover the correct magnitude of the underlying RI distribution.

Overall, the reconstruction demonstrates stable convergence and reasonable qualitative agreement with the target measurements for this synthetic bead phantom. The observed amplitude discrepancy highlights a limitation of the current formulation and suggests that further improvements in the conditioning of the inverse problem may be required to achieve a fully quantitative

reconstruction.

5 Conclusion and Discussion

This work presents a WebGPU-based framework for forward and inverse modeling in IDT, supporting multiple propagation models and an iterative reconstruction pipeline within a unified GPU backend. Experimental results demonstrate that the forward models achieve high numerical accuracy in efficient time and that the reconstruction procedure is capable of recovering the underlying structure of synthetic samples with stable convergence.

A key contribution of this work is the emphasis on accessibility and portability. By leveraging WebGPU and compiling the backend to WebAssembly, the system can be executed across a wide range of hardware environments, including within a browser, without reliance on specialized GPU frameworks such as CUDA. This significantly lowers the barrier to entry for computational imaging workflows and enables more reproducible and widely deployable systems. In particular, this opens the door to interactive and educational use cases, where users can explore physics-based imaging models directly in the browser without requiring complex setup or dedicated hardware.

At the same time, the performance evaluation highlights important limitations of the current approach. While the WebGPU implementation exhibits correct scaling behavior, it remains slower than optimized CUDA-based implementations, with runtime dominated by memory movement, per-slice computation, and execution overhead rather than individual algorithmic components. Additionally, browser-based execution introduces practical constraints on available memory, limiting the size of volumes that can be processed compared to native GPU environments. The current browser-based demonstration is limited to forward simulation, as the full reconstruction pipeline remains computationally intensive for real-time execution within the constraints of a web environment.

Overall, this work demonstrates that complex computational imaging pipelines can be implemented in a portable, hardware-agnostic framework. While further optimization is needed to close the performance gap with native implementations, the results establish WebGPU as a viable platform for accessible and reproducible computational imaging systems.

References

[1] R. Ling, W. Tahir, H.-Y. Lin, H. Lee, and L. Tian, "High-throughput intensity diffraction tomography with a computational microscope," *Biomedical Optics Express*, vol. 9, p. 2130, Apr. 2018.

[2] J. Zhu, H. Wang, and L. Tian, "High-fidelity intensity diffraction tomography with a non-paraxial multiple-scattering model," *Optics Express*, vol. 30, p. 32808, Aug. 2022.

[3] E. Michel, "Webgpu-cpp: A single-file c++ idiomatic wrapper for webgpu." `https://github.com/eliemichel/WebGPU-Cpp`, 2023.

[4] M. Feit and J. Fleck, "Light propagation in graded-index optical fibers," *Applied Optics*, vol. 17, pp. 3990–3998, 12 1978.

[5] M. Born and E. Wolf, *Principles of Optics: Electromagnetic Theory of Propagation, Interference and Diffraction of Light*. Cambridge University Press, 7th ed., 1999.